KAOP • AGENTIC OPERATIONS

AI agents move fast.

Can you govern them?

Bring incident response and code review into one accountable agent operating model.



A customer conversation for Platform Engineering leaders

Your next incident will cross agent boundaries.

Independent agents may work well. Shared accountability gets harder.

01

NO SHARED INVENTORY

Which agents are active, who owns them, and what are they doing?

02

INCONSISTENT ACCESS

Different tools, credentials and permissions create an uneven risk surface.

03

NO END-TO-END TRAIL

The alert, agent findings and code review become separate stories no one can audit.

The platform engineering question: **Who authorized it? What happened? Can we prove it?**

One operating model for your agents.

Build, run, govern and optimize workflows on shared production context.

BUILD

Bring existing agents, skills or runbooks.

RUN

Coordinate triggers, tools and specialist agents.

GOVERN

Scope actions, enforce gates and record decisions.

OPTIMIZE

Evaluate quality, reliability and cost over time.

ONE SHARED FOUNDATION Context • Memory • Integrations • Policy • Run History

Trust is enforced at the action boundary.

A clever prompt is not a permission model.

01 **Identity and scoped permissions**

02 **Allowed data sources and tools**

03 **Policy check and human approval**

04 **Recorded actions and outcomes**

PROPOSED DEMO POLICY

Incident agent

Read pods, events and logs.
No deploys or pod deletion.

PR reviewer

Read diff, post a review.
No merge or deployment authority.

Proposed least-privilege scopes; enforce and verify inside the KAOP workspace.

One incident. Two workflows. One control plane.

Target workflows: separate triggers; shared controls verified after setup.

01 INCIDENT INVESTIGATION

Grafana alert

Grafana fires → **KAOP routes** → **K8s RCA** → **Evidence + fix**

02 CODE REVIEW

Fresh GitHub PR

PR opens → **KAOP routes** → **Reviewer** → **Comment**

SHARED CONTROLS Identity • Permissions • Tool policy • Human gates • Audit trail

LIVE | One fault. Follow every decision.

Watch a Redis outage become an API restart, then a reviewable engineering fix.

01

HEALTHY

API ↔ Redis
probes correct

→

02

FAULT

Probe points to
/readyz; Redis off

→

03

DETECT

HTTP 503 + restart
Grafana fires

→

04

INVESTIGATE

KAOP RCA
evidence + fix

→

05

REVIEW

Human opens PR
agent comment

WHAT THE CUSTOMER SEES **Evidence → Root cause → Controlled action → Recorded outcome**

Historical lab: 2 restarts + Grafana Alerting. KAOP verification pending.

Start with one owner and a measurable pilot.

Two-week pilot: one owned workflow, scoped authority and agreed evidence.

01

PROVE

One alert → credible RCA

One PR → useful review

02

CONTROL

Tool reach is explicit

Gate higher-risk actions

03

SCALE

Adopt the operating standard

Measure quality, speed and cost

Pilot scorecard

Time to useful RCA • Review usefulness • Policy adherence • Cost per run

Can we agree one workflow, one owner and a two-week pilot?